Library Management System

Seif Elden Waleed 192300230

Martin Milad 192300344

Mina Ehab 192300365

David Alber 192300384

Nour Romany 192300393

Nicole Aziz 192300428

David Gerges 192300429

No.	Achievement	Status (Yes/No)
1	Stack correctly implemented	
2	Queue correctly implemented	
3	Linked List correctly implemented	
4	Core functionality working	
5	Additional features (if any) implemented	
6	Code is clean and well-commented	
7	Report completed	

Prepared for:

Dr. Fatma sakr

Assistant Lecturer : **Doaa Mabrouk**

T/A : **Hussen Moahmed**

1. Overview

This project implements a simplified Library Management System in C++ using **manual implementations** of fundamental data structures: Stack, Queue, and Linked List. The system allows for adding, borrowing, returning, and displaying books.

2. Data Structures Used

a. Linked List – Catalog

- **Purpose**: Maintains a dynamic list of all books in the library.
- **Class**: `Catalog`
- **Node Structure**: Each node (`Book`) stores `id`, `title`, and a pointer to the next book.
- **Operations**:
  - ***`addBook(int id, string title)`*** – Adds a new book to the end of the list. ○ ***`searchBook(int id)`*** – Searches a book by ID. ○ ***`display()`*** – Displays all books in the catalog.

b. Queue – Borrowed Books

- **Purpose**: Keeps track of borrowed books in the order they were borrowed (FIFO).
- **Class**: `Queue`
- **Implementation**: Array of size 10 with `front` and `rear` pointers.
- **Operations**:
  - ***`enqueue(int bookId)`*** – Adds a book to the borrow queue. ○ ***`dequeue()`*** – Removes the oldest borrowed book. ○ ***`isBookBorrowed(int bookId)`*** – Checks if a book is currently borrowed. ○ ***`remove(int bookId)`*** – Removes a specific book from the borrow queue. ○ ***`display()`*** – Lists all currently borrowed books.

c. Stack – Returned Books

- **Purpose**: Tracks the most recently returned books (LIFO).
- **Class**: `Stack`
- **Implementation**: Array of size 10 with `top` index.
- **Operations**:
  - ***`push(int bookId)`*** – Adds a returned book to the stack.

- ***pop()*** – Removes the most recently returned book. ○ ***display()*** – Lists all returned books (most recent on top).

---

3. Sample Outputs

1. Add Books:

Input:

Enter your choice: **1**

Enter Book ID: **101**

Enter Book Title: **Data Structure**

Output:

Added book: **[101] Data Structure**

```
===== Library Management Menu =====
1. Add New Book
2. Search Book by ID
3. Borrow Book by ID
4. Return Book by ID
5. Display Catalog
6. Display Borrow Queue
7. Display Return Stack
0. Exit
Enter your choice: 1
Enter Book ID: 101
Enter Book Title: Data Structure
Added book: [101] Data Structure
```

Input:

Enter your choice: **1**

Enter Book ID: **102**

Enter Book Title: **Software Engineering**

Output:

Added book: **[102] Software Engineering**

```
===== Library Management Menu =====
1. Add New Book
2. Search Book by ID
3. Borrow Book by ID
4. Return Book by ID
5. Display Catalog
6. Display Borrow Queue
7. Display Return Stack
0. Exit
Enter your choice: 1
Enter Book ID: 102
Enter Book Title: Software Engineering
Added book: [102] Software Engineering
```

2. Display Catalog:

Input:

Enter your choice: **5**

Output:

Library Catalog:

[101] **Data Structure**

[102] **Software Engineering**

```
===== Library Management Menu =====
1. Add New Book
2. Search Book by ID
3. Borrow Book by ID
4. Return Book by ID
5. Display Catalog
6. Display Borrow Queue
7. Display Return Stack
0. Exit
Enter your choice: 5
Library Catalog:
  [101] Data Structure
  [102] Software Engineering
```

3.Search Book by ID:

Input:

Enter your choice: **2**

Enter Book ID to search: **102**

Output:

Found: **[102] Software Engineering**

```
===== Library Management Menu =====
1. Add New Book
2. Search Book by ID
3. Borrow Book by ID
4. Return Book by ID
5. Display Catalog
6. Display Borrow Queue
7. Display Return Stack
0. Exit
Enter your choice: 2
Enter Book ID to search: 102
Found: [102] Software Engineering
```

4.Borrow Book by ID:

Input:

Enter your choice: **3**

Enter Book ID to borrow: **102**

Output:

Borrow request for Book ID **102** added.

```
===== Library Management Menu =====
1. Add New Book
2. Search Book by ID
3. Borrow Book by ID
4. Return Book by ID
5. Display Catalog
6. Display Borrow Queue
7. Display Return Stack
0. Exit
Enter your choice: 3
Enter Book ID to borrow: 102
Borrow request for Book ID 102 added.
```

5.Display Borrow Queue:

Input:

Enter your choice: **6**

Output:

Borrow Queue:

Book ID: **102**

```
===== Library Management Menu =====
1. Add New Book
2. Search Book by ID
3. Borrow Book by ID
4. Return Book by ID
5. Display Catalog
6. Display Borrow Queue
7. Display Return Stack
0. Exit
Enter your choice: 6
Borrow Queue:
  Book ID: 102
```

6. Return Book by ID:

Input:

Enter your choice: **4** Enter Book ID to return:**102**

Output:

Book ID 102 has been successfully returned.

```
===== Library Management Menu =====
1. Add New Book
2. Search Book by ID
3. Borrow Book by ID
4. Return Book by ID
5. Display Catalog
6. Display Borrow Queue
7. Display Return Stack
0. Exit
Enter your choice: 4
Enter Book ID to return: 102
Book ID 102 has been successfully returned.
```

7.Display Return Stack:

Input:

Enter your choice: **7**

Output:

Return Stack:

Book ID: **102**

```
===== Library Management Menu =====
1. Add New Book
2. Search Book by ID
3. Borrow Book by ID
4. Return Book by ID
5. Display Catalog
6. Display Borrow Queue
7. Display Return Stack
0. Exit
Enter your choice: 7
Return Stack:
  Book ID: 102
```

4. Suggested Improvements

- **Dynamic Data Structures**: Replace fixed-size arrays in `Stack` and `Queue` with dynamically allocated structures to handle more books.
- **Book Details on Return/Borrow**: Store the entire book object in `Queue` and `Stack` instead of just `bookId`, to show book titles too.
- **Persistent Storage**: Implement file input/output to save catalog and transaction history between program runs.
- **User Roles**: Add different roles like admin and member with access control.
- **GUI Interface**: Implement a graphical user interface using Qt or a web framework for better usability.

5. Conclusion

- This project demonstrates the use of core data structures in building a functional Library Management System. Manual implementation enhanced understanding of stack, queue, and linked list mechanics. The system meets basic requirements and is a strong foundation for future enhancements.

Code

Catalog.h

```
#pragma once
#include <string>
using namespace std;

// Book node for the catalog linked list
struct Book {
    int id;
    string title;
    Book* next;
};

class Catalog {
private:
    Book* head; // Start of the catalog list

public:
    Catalog(); // Constructor

    void addBook(int id, string title); // Add a book to the catalog

    Book* searchBook(int id); // Search for a book by ID

    void display(); // Display all books in the catalog
};
```

Queue.h

```
#pragma once

// Queue for managing borrowed book IDs
class Queue {
private:
    int arr[10];             // Fixed-size array for queue
    int front = -1, rear = -1; // Queue pointers

public:
    void enqueue(int bookId);       // Add book ID to queue
    int dequeue();                  // Remove and return front book ID
    void display();                 // Show all book IDs in queue
    bool isBookBorrowed(int bookId); // Check if book ID is in queue
    bool remove(int bookId);        // Remove specific book ID
};
```

Stack.h

```
#pragma once

// Stack class to store returned book IDs
class Stack {
private:
    int arr[10];  // Array to store book IDs
    int top;      // Index of the top element

public:
    Stack();                // Constructor to initialize stack
    void push(int bookId); // Push a book ID onto the stack
    int pop();              // Pop the top book ID from the stack
    void display();         // Display all returned book IDs in the stack
};
```

Catalog.cpp

```
#include "Catalog.h"
#include <iostream>
using namespace std;

// Constructor: Initializes the catalog as empty
Catalog::Catalog() {
    head = nullptr;
}

// Adds a new book with given ID and title to the end of the catalog
void Catalog::addBook(int id, string title) {
    Book* newBook = new Book{ id, title, nullptr };

    if (head == nullptr) {
        head = newBook;
    }
    else {
        Book* temp = head;
        while (temp->next != nullptr) {
            temp = temp->next; // Traverse to the end
        }
        temp->next = newBook; // Append the new book
    }

    cout << "Added book: [" << id << "] " << title << "\n";
}

// Searches for a book by ID and returns a pointer to it (or nullptr if not found)
Book* Catalog::searchBook(int id) {
    Book* temp = head;
    while (temp != nullptr) {
        if (temp->id == id) {
            return temp; // Book found
        }
```

```
        temp = temp->next;
    }
    return nullptr; // Not found
}

// Displays all books in the catalog
void Catalog::display() {
    Book* temp = head;
    cout << "Library Catalog:\n";

    if (temp == nullptr) {
        cout << "  (empty)\n";
        return;
    }

    while (temp != nullptr) {
        cout << "  [" << temp->id << "] " << temp->title << "\n";
        temp = temp->next;
    }
}
```

Queue.cpp

```
#include "Queue.h"
#include<iostream>
using namespace std;

void Queue::enqueue(int bookId) {
    // Check if there's space to add a new book
    if (rear < 9) {
        if (front == -1) {
            front = 0;  // If queue is empty, set front to 0
        }
        rear++;  // Move rear to next position
        arr[rear] = bookId;  // Add book ID to the queue
        cout << "Borrow request for Book ID " << bookId << " added.\n";
    }
    else {
        cout << "Borrow queue is full.\n";  // Queue is full
    }
}

int Queue::dequeue() {
    // Check if queue is empty
    if (front == -1 || front > rear) {
        cout << "Borrow queue is empty.\n";
        return -1;  // Return -1 if queue is empty
    }
    else {
        return arr[front++];  // Remove book from front and return its ID
    }
}

void Queue::display() {
    cout << "Borrow Queue:\n";
    if (front == -1 || front > rear) {
```

```
        cout << "  (empty)\n";  // If queue is empty, display empty message
        return;
    }
    for (int i = front; i <= rear; i++)
        cout << "  Book ID: " << arr[i] << "\n";  // Display each book ID
}

bool Queue::isBookBorrowed(int bookId) {
    for (int i = front; i <= rear; ++i) {
        if (arr[i] == bookId) {
            return true;  // Return true if book is in the queue
        }
    }
    return false;  // Return false if book is not found
}

bool Queue::remove(int bookId) {
    if (front == -1 || front > rear) {
        return false;  // Return false if queue is empty
    }

    bool found = false;
    int newRear = front - 1;

    for (int i = front; i <= rear; i++) {
        if (arr[i] == bookId) {
            found = true;  // Mark book for removal
            continue;
        }
        newRear++;  // Move books that aren't removed
        arr[newRear] = arr[i];
    }

    rear = newRear;  // Update rear after removal
    if (rear < front) {
        front = -1;
        rear = -1;  // Reset queue if empty
    }
    return found;  // Return whether the book was removed
}
```

Stack.cpp

```
#include "Stack.h"
#include <iostream>
using namespace std;

// Constructor: initialize stack
Stack::Stack() {
    top = -1;
}

// Push book ID onto the stack
void Stack::push(int bookId) {
    if (top < 9) {
        top++;
```

```
        arr[top] = bookId;
        cout << "Book ID " << bookId << " has been successfully returned.\n";
    }
    else {
        cout << "Return stack is full.\n";
    }
}

// Pop book ID from the stack
int Stack::pop() {
    if (top == -1) {
        cout << "Return stack is empty.\n";
        return -1;
    }
    return arr[top--];
}

// Display all returned book IDs in the stack
void Stack::display() {
    cout << "Return Stack:\n";
    if (top == -1) {
        cout << "  (empty)\n";
        return;
    }

    for (int i = top; i >= 0; i--)
        cout << "  Book ID: " << arr[i] << "\n";
}
```

Main

```
#include <iostream>
#include "Catalog.h"
#include "Queue.h"
#include "Stack.h"
using namespace std;

int main() {
    Catalog catalog;
    Queue borrowQueue;
    Stack returnStack;

    int choice;
    do {
        // Main menu
        cout << "\n===== Library Management Menu =====\n";
        cout << "1. Add New Book\n";
        cout << "2. Search Book by ID\n";
        cout << "3. Borrow Book by ID\n";
        cout << "4. Return Book by ID\n";
        cout << "5. Display Catalog\n";
        cout << "6. Display Borrow Queue\n";
        cout << "7. Display Return Stack\n";
        cout << "0. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;
```

```
        int id;
        string title;

        switch (choice) {
        case 1:
            // Add new book
            cout << "Enter Book ID: ";
            cin >> id;
            cin.ignore();
            cout << "Enter Book Title: ";
            getline(cin, title);
            catalog.addBook(id, title);
            break;
        case 2:
            // Search book by ID
            cout << "Enter Book ID to search: ";
            cin >> id;
            if (Book* book = catalog.searchBook(id))
                cout << "Found: [" << book->id << "] " << book->title << "\n";
            else
                cout << "Book not found.\n";
            break;
        case 3:
            // Borrow book
            cout << "Enter Book ID to borrow: ";
            cin >> id;
            if (catalog.searchBook(id))
                borrowQueue.enqueue(id);
            else
                cout << "Book ID not found in catalog.\n";
            break;
        case 4:
            // Return book
            cout << "Enter Book ID to return: ";
            cin >> id;

            if (borrowQueue.isBookBorrowed(id)) {
                returnStack.push(id);
                borrowQueue.remove(id); // Remove from borrow queue
            }
            else {
                cout << "Book ID " << id << " was not borrowed and cannot be
returned.\n";
            }
            break;
        case 5:
            // Show catalog
            catalog.display();
            break;
        case 6:
            // Show borrow queue
            borrowQueue.display();
            break;
        case 7:
            // Show return stack
            returnStack.display();
```

```
            break;
        case 0:
            // Exit
            cout << "Exiting program.\n";
            break;
        default:
            cout << "Invalid choice. Try again.\n";
        }
    } while (choice != 0);

    return 0;
}
```